

Домашнє завдання 06 — Мережі. Частина 3

Автор: Андрій Пономарьов

Середовище: macOS, Wireshark 4.6.7

Інструменти: dig, route, traceroute, curl, tshark, openssl; Claude Code — редактура

Завдання: наскрізний розбір шляху до сайту — dig (A, AAAA, NS, SOA) → route → traceroute; дамп трафіку під час curl -v і в Wireshark знайти TCP handshake, ClientHello з SNI та першу відповідь; виміряти фази curl -w (DNS, TCP, TLS, перший байт) і визначити найдовшу; openssl s_client — хто підписав сертифікат, скільки днів лишилось, чи відданий весь ланцюжок. Опціонально: оцінка на sslabs.com, порівняння з конфігом ssl-config.mozilla.org, 5 речень про власний сервіс.

Обраний сайт: **ithillel.ua** (сайт школи; у ДЗ 4 був github.com — цього разу цікавіший кейс: DNS на Cloudflare, а хостинг у Hetzner).

Примітка: робота виконувалась на macOS, тому замість ip route get — нативний route -n get . Дамп знято tshark'ом (консольний Wireshark), аналіз — у Wireshark GUI.

Коротко про результат

Пункт	Результат
DNS і маршрут	ithillel.ua → Hetzner; DNS — Cloudflare; локально доступний лише IPv4-маршрут
Traceroute	12 хопів ICMP, кінцева затримка близько 40–45 мс
Wireshark	TCP handshake — кадри 1–3; ClientHello з SNI — кадр 4; ServerHello — кадр 8
Продуктивність	DNS 6 мс; TCP +38 мс; TLS +43 мс; TTFB +44 мс; тіло +210 мс
Сертифікат	Let's Encrypt YR2, 72 дні до завершення; переданий повний потрібний ланцюжок
TLS-профіль	SSL Labs A+; TLS 1.2/1.3; є застарілі CBC-набори для TLS 1.2

1. dig — чотири типи записів

```
$ dig A      ithillel.ua  → 78.47.64.83          (TTL 228)
$ dig AAAA  ithillel.ua  → 2a01:4f8:c012:dc97::1 (TTL 238)
$ dig NS    ithillel.ua  → coco.ns.cloudflare.com, norm.ns.cloudflare.com
$ dig SOA   ithillel.ua  → coco.ns.cloudflare.com. dns.cloudflare.com.
                               2408945412 10000 2400 604800 1800
```

Що це означає:

- **A** — сайт живе на 78.47.64.83. За whois це **Hetzner Online**, дата-центр Falkenstein (Німеччина).

- **AAAA** — є й IPv6-адреса, 2a01:4f8:... — той самий Hetzner Cloud. Тобто сервер доступний по обох протоколах.
- **NS** — зону обслуговують сервери **Cloudflare**. При цьому А-запис віддає прямий IP Hetzner, а не 104.x: DNS на Cloudflare є, а проксі-режим («помаранчева хмаринка») вимкнено — трафік іде напряму на сервер.
- **SOA** — «паспорт зони»: первинний NS (coco.ns.cloudflare.com), контакт адміна (dns.cloudflare.com → dns@cloudflare.com), серійний номер зони 2408945412 і таймери: refresh 10000 с (як часто secondary перевіряє оновлення), retry 2400 с, expire 604800 с (тиждень — і secondary перестав відповідати за зону), negative TTL 1800 с (скільки кешується відповідь «такого запису немає»).

Відповідав локальний резолвер 10.255.255.1, А-запит — 18 мс, решта — 5–9 мс (зона вже в кеші). Повний вивід: [docs/dig-output.txt](#).

2. route — куди піде пакет

```
$ route -n get ithillel.ua
route to: 78.47.64.83
gateway: 10.20.50.1
interface: en0
mtu: 1500

$ route -n get -inet6 2a01:4f8:c012:dc97::1
route: writing to routing socket: not in table
```

Що це означає: IPv4-шлях звичайний — адреса не локальна, пакет піде через шлюз 10.20.50.1 з Wi-Fi-інтерфейсу en0. А от IPv6-запит показовий: AAAA-запис у сайті є, але маршруту до нього в перевірених конфігурації **немає** — рівно той висновок, що й у ДЗ 5 (локально є лише link-local та ULA). Тому `curl -v` нижче чесно пише IPv6: (none) і йде по IPv4. Повний вивід: [docs/route-output.txt](#).

3. traceroute — шлях пакета

```
$ traceroute -I -n -w 2 -m 20 ithillel.ua
 1  10.20.50.1      3.0 ms  ← мій роутер
 2  10.12.18.1     9.8 ms  ← приватна мережа провайдера
 3  10.1.1.3       4.5 ms
 4  91.226.254.3   5.4 ms  ← публічна адреса провайдера (IPORT)
 5  185.1.63.223  46.8 ms ← обмінна точка Giganet global IX
 6  213.239.224.218 47.5 ms ← магістраль Hetzner (AS24940)
 7  213.239.224.181 39.9 ms
 8  * 213.239.227.197 201 ms
 9-10 * * *
11 136.243.181.18 39.5 ms ← core-поутер ДЦ Falkenstein (fsn1-dc8)
12 78.47.64.83    45.4 ms ← сервер ithillel.ua
```

Що це означає: до сайту — **12 хопів**. Картина та сама, що в ДЗ 4: перший хоп — домашній роутер, 2–3 — приватні адреси провайдера, з 4-го видно публічні адреси.

Стрибок затримки ~5 → ~46 мс на хопі 5 збігається з переходом на міжоператорський маршрут через обмінну точку; далі видно адреси Hetzner і кінцевий сервер. За одним traceroute не можна точно визначити географічну межу. UDP-варіант після 11-го хопу отримує лише *, тоді як ICMP (– I) доходить до сервера; це свідчить про різну фільтрацію або політику відповідей уздовж маршруту, але не локалізує її на самому сервері. Повний вивід обох варіантів: [docs/traceroute-output.txt](#).

4. Дамп під час curl -v: handshake, ClientHello з SNI, перша відповідь

Захоплення tshark'ом на en0 з фільтром `host 78.47.64.83`, паралельно — `curl -v https://ithillel.ua/`. Дамп: [docs/curl-ithillel.pcapng](#), вивід curl: [docs/curl-v-output.txt](#).

TCP handshake (кадри 1–3)

```
1 0.000000 10.20.50.89 → 78.47.64.83 [SYN, ECE, CWR] Seq=0 MSS=1460
2 0.039102 78.47.64.83 → 10.20.50.89 [SYN, ACK, ECE] Seq=0 Ack=1 MSS=1460
3 0.039298 10.20.50.89 → 78.47.64.83 [ACK] Seq=1 Ack=1
```

Той самий three-way handshake, що в ДЗ 4, тільки тепер у Wireshark: SYN → SYN-ACK → ACK, рівно один RTT ≈ 39 мс (збігається з traceroute).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.20.50.89	78.47.64.83	TCP	78	51631 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=943710973 TSecr=0 SACK_PERM=0
2	0.039102	78.47.64.83	10.20.50.89	TCP	74	443 → 51631 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM=0 TSval=2788008368 TSecr=943710973
3	0.039298	10.20.50.89	78.47.64.83	TCP	66	51631 → 443 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=943711013 TSecr=2788008368
4	0.039558	10.20.50.89	78.47.64.83	TLSv1.3	387	Client Hello (SNI=ithillel.ua)
5	0.080167	78.47.64.83	10.20.50.89	TCP	66	443 → 51631 [ACK] Seq=1 Ack=322 Win=64896 Len=0 TSval=2788008408 TSecr=943711013
6	0.080170	78.47.64.83	10.20.50.89	TCP	1514	[TCP Previous segment not captured] 443 → 51631 [PSH, ACK] Seq=1449 Ack=322 Win=64896 Len=1448 TSval=2788008408 TSecr=943711013
7	0.080172	78.47.64.83	10.20.50.89	TCP	1266	443 → 51631 [PSH, ACK] Seq=2897 Ack=322 Win=64896 Len=1200 TSval=2788008408 TSecr=943711013 [TCP PD
8	0.080174	78.47.64.83	10.20.50.89	TLSv1.3	1514	[TCP Retransmission] , Server Hello, Change Cipher Spec, Application Data
9	0.080372	10.20.50.89	78.47.64.83	TCP	78	[TCP Dup ACK 3#1] 51631 → 443 [ACK] Seq=322 Ack=4097 Win=131072 Len=0 TSval=943711054 TSecr=2788008408
10	0.080414	10.20.50.89	78.47.64.83	TCP	66	51631 → 443 [ACK] Seq=322 Ack=4097 Win=127680 Len=0 TSval=943711054 TSecr=2788008408
11	0.080914	10.20.50.89	78.47.64.83	TCP	66	[TCP Window Update] 51631 → 443 [ACK] Seq=322 Ack=4097 Win=131072 Len=0 TSval=943711054 TSecr=2788008408
12	0.081225	78.47.64.83	10.20.50.89	TLSv1.3	607	Application Data, Application Data, Application Data
13	0.081289	10.20.50.89	78.47.64.83	TCP	66	51631 → 443 [ACK] Seq=322 Ack=4638 Win=130560 Len=0 TSval=943711055 TSecr=2788008410
14	0.082544	10.20.50.89	78.47.64.83	TLSv1.3	72	Change Cipher Spec
15	0.082571	10.20.50.89	78.47.64.83	TLSv1.3	140	Application Data
16	0.082795	10.20.50.89	78.47.64.83	TLSv1.3	152	Application Data
17	0.082947	10.20.50.89	78.47.64.83	TLSv1.3	125	Application Data
18	0.121881	78.47.64.83	10.20.50.89	TCP	66	443 → 51631 [ACK] Seq=4638 Ack=402 Win=64896 Len=0 TSval=2788008450 TSecr=943711056
19	0.121884	78.47.64.83	10.20.50.89	TLSv1.3	353	Application Data
20	0.121886	78.47.64.83	10.20.50.89	TLSv1.3	353	Application Data
21	0.121887	78.47.64.83	10.20.50.89	TLSv1.3	128	Application Data
22	0.122365	10.20.50.89	78.47.64.83	TCP	66	51631 → 443 [ACK] Seq=547 Ack=5274 Win=130496 Len=0 TSval=943711096 TSecr=2788008450
23	0.123223	10.20.50.89	78.47.64.83	TLSv1.3	97	Application Data
24	0.124112	78.47.64.83	10.20.50.89	TCP	66	443 → 51631 [ACK] Seq=5274 Ack=547 Win=64768 Len=0 TSval=2788008454 TSecr=943711056
25	0.125781	78.47.64.83	10.20.50.89	TLSv1.3	371	Application Data
26	0.125784	78.47.64.83	10.20.50.89	TCP	1514	443 → 51631 [ACK] Seq=5579 Ack=547 Win=64768 Len=1448 TSval=2788008454 TSecr=943711056 [TCP PDU rec

Frame 1: Packet, 78 bytes on wire (624 bits), 78 bytes captured (624 bits) on interface en0

Ethernet II, Src: 46:c0:a4:4b:9b:6b (46:c0:a4:4b:9b:6b), Dst: Google_43:f3:c8 (b0:e4:d5:43:f3:c8)

Internet Protocol Version 4, Src: 10.20.50.89, Dst: 78.47.64.83

Transmission Control Protocol, Src Port: 51631, Dst Port: 443, Seq: 0, Len: 0

0000 b0 e4 d5 43 f3 c8 46 c0 a4 4b 9b 6b 08 00 45 00 ...C..F..K.k..E..

0010 00 40 00 00 00 00 40 06 af c9 0a 14 32 59 de 2f @...@...2YN/

0020 40 53 c9 af 01 bb ba 3b 02 1e 00 00 00 00 b0 c2 @S.....

0030 ff ff c8 4b 00 00 02 04 05 b4 01 03 03 06 01 01 ...K.....

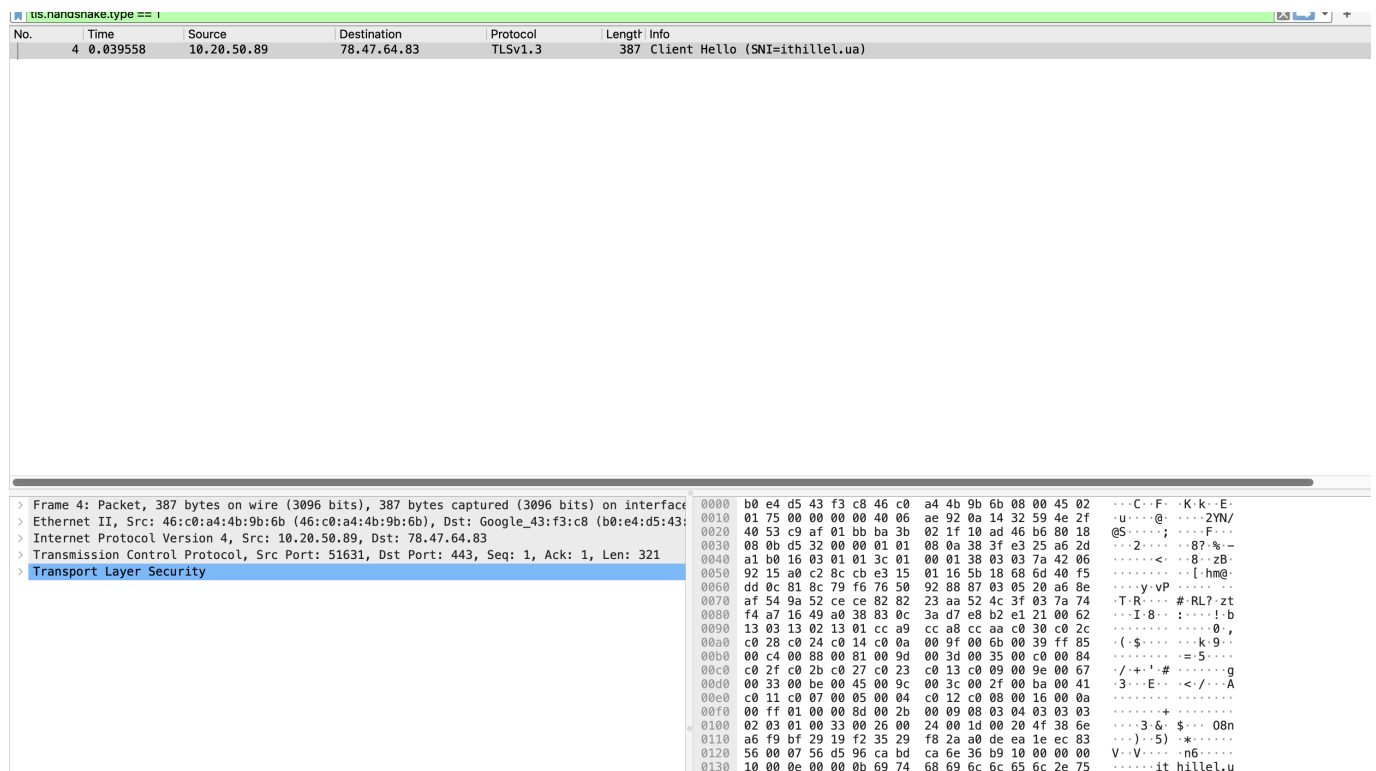
0040 08 0a 38 3f e2 fd 00 00 00 00 02 00 00 ..8?.....

ClientHello з SNI (кадр 4)

Одразу після handshake (0.0396 с) клієнт шле перший TLS-запис — Client Hello. Wireshark просто в колонці Info показує головне: `Client Hello (SNI=ithillel.ua)`. У деталях розширення `server_name`:

```
Extension: server_name (len=16)
  Server Name Indication extension
    Server Name Type: host_name (0)
    Server Name: ithillel.ua
```

Навіщо SNI: ім'я хоста передається **відкритим текстом до** того, як з'явиться шифрування — інакше сервер, на якому живе кілька сайтів, не знав би, який сертифікат показати. У hex-панелі скріншота видно й сам рядок `ithillel.ua`, і ALPN-пропозицію `h2 http/1.1`:

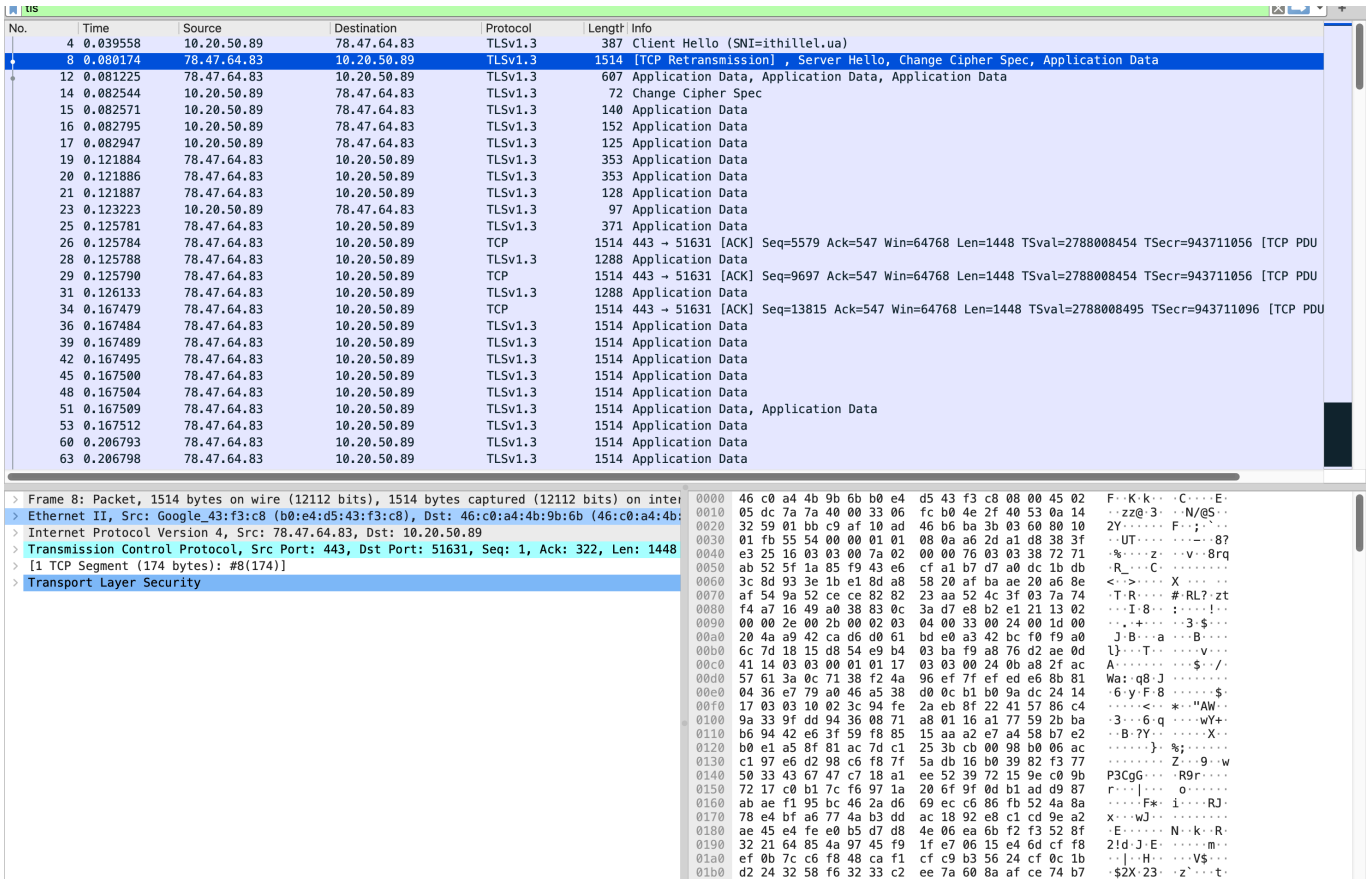


ServerHello і перша відповідь

Відповідь сервера (кадр 8, 0.0802 с): Server Hello, Change Cipher Spec, Application Data — сервер обрав **TLS 1.3** і шифр **TLS_AES_256_GCM_SHA384**. Wireshark показує **Previous segment not captured**, **SACK** і **TCP Retransmission**: початковий сегмент не потрапив у дамп або був побачений не по порядку. Оскільки всі ці кадри мають практично однаковий timestamp, дамп не дає підстав упевнено назвати це реальною втратою в мережі. Для складання TLS із таких сегментів увімкнено `tcp.reassemble_out_of_order`. У TLS 1.3 сертифікат летить уже зашифрованим (**Application Data** в тому ж кадрі) — на відміну від TLS 1.2, де Certificate було видно відкрито.

Перша **HTTP-відповідь** приходить на 0.12–0.13 с — у дампі це перші кадри **Application Data** від сервера після мого запиту (розшифрувати їх Wireshark не може, ключі лишились у curl), а розшифровану версію показує сам `curl -v`:

```
< HTTP/2 200
< server: nginx
< content-length: 651659
< strict-transport-security: max-age=31536000; includeSubDomains; preload
```



Текстовий розбір дампа: [docs/tshark-analysis.txt](#).

5. curl -w — фази запиту

Три запуски поспіль ([docs/curl-timing-output.txt](#)):

Фаза	Запуск 1	Запуск 2	Запуск 3	Що вимірює
DNS (namelookup)	63 мс	6 мс	8 мс	резолв імені
TCP (connect)	104 мс	44 мс	48 мс	+ handshake SYN/SYN-ACK/ACK
TLS (appconnect)	149 мс	88 мс	93 мс	+ TLS-handshake
Перший байт (starttransfer)	190 мс	131 мс	135 мс	+ запит і відповідь nginx
Всього (total)	402 мс	342 мс	344 мс	+ завантаження тіла 651 КБ

Що найдовше. Дивитись треба на *прирости*, а не на абсолютні цифри (кожен рядок включає попередні). У «теплому» запуску 2: DNS 6 мс, TCP +38 мс, TLS +43 мс, перший байт +44 мс, тіло +210 мс. Тобто:

- з фаз встановлення з'єднання найдовша — **TLS** (~43 мс), але це рівно один RTT: TLS 1.3 вкладає весь handshake в один обмін (у TLS 1.2 було б два RTT ≈ 80 мс);
- TCP і очікування першого байта — теж по ~одному RTT ≈ 40 мс: на такій відстані (Німеччина) усе визначає швидкість світла, а не CPU;
- DNS із кешем майже безплатний (6 мс), «холодний» — 63 мс (запуск 1);

- а найдовша частина запиту взагалі — **завантаження тіла** (~210 мс, 651 КБ HTML): сумарно довше, ніж усі handshake разом.

6. openssl s_client — сертифікат і ланцюжок

```
$ openssl s_client -connect ithillel.ua:443 -servername ithillel.ua -showcerts
---
Certificate chain
 0 s:CN=ithillel.ua          i:C=US, O=Let's Encrypt, CN=YR2
 1 s:C=US, O=Let's Encrypt, CN=YR2 i:C=US, O=ISRG, CN=Root YR
 2 s:C=US, O=ISRG, CN=Root YR      i:C=US, O=Internet Security Research Group, CN=ISRG Root X1
---
Verification: OK
Protocol: TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384

subject=CN=ithillel.ua (SAN: *.ithillel.ua, ithillel.ua)
notBefore=Jul  7 20:29:50 2026 GMT
notAfter=Oct  5 20:29:49 2026 GMT
```

Відповіді на питання завдання:

- **Хто підписав:** Let's Encrypt, проміжний CA **YR2** (нове покоління проміжних LE; ланцюжок веде до кореня ISRG Root YR, який для старих клієнтів крос-підписаний класичним ISRG Root X1).
- **Скільки днів лишилось:** сертифікат виданий 07.07.2026 до 05.10.2026 — типовий 90-денний строк Let's Encrypt; на момент перевірки (25.07.2026) лишилось **72 дні**. Спосіб продовження із сертифіката визначити не можна.
- **Чи весь ланцюжок віддано:** так — сервер шле **3 сертифікати**: свій (leaf) і обидва проміжні (YR2, Root YR). Кореневий не шле — і не має: корінь клієнт і так тримає у своєму сховищі довіри, тому **Verification: OK**. Якби сервер віддав лише leaf, частина клієнтів не змогла б добудувати ланцюжок і лаялась би на «unknown issuer».

Повний вивід: [docs/openssl-output.txt](#), підсумок по сертифікату: [docs/cert-summary.txt](#).

Опціональна частина: безпека

7. SSL Labs — оцінка A+

Скан через [api.ssllabs.com](#) (обидва ендпоінти — IPv4 і IPv6): **A+**, без жодного warning. Підсумок: [docs/ssllabs-summary.txt](#).

Що підтвердив скан: сервер підтримує лише TLS 1.2 і 1.3, усі погоджені набори мають forward secrecy (ECDHE), а HSTS налаштовано як `max-age=31536000; includeSubDomains; preload`. За сукупністю перевірок обидва ендпоінти отримали A+.

Що можна поліпшити, попри оцінку A+:

- у TLS 1.2 досі увімкнені 4 **СВС-шифри** (`...AES_256_CBC_SHA` тощо) — SSL Labs позначає їх «weak»: режим CBC у TLS історично вразливий до padding-атак (Lucky13 і родина);

- ключ **RSA 2048** — прийнятний, хоча ECDSA P-256 зазвичай дає менший сертифікат і швидші операції;
- **OCSP stapling** **вимкнено** — раніше за це теж чіпляли, хоча зараз це вже не мінус (див. нижче).

8. Порівняння з конфігом `ssl-config.mozilla.org`

Згенерований конфіг (nginx, профіль **Intermediate**, guidelines 6.0): [docs/mozilla-nginx-intermediate.conf](https://docs.mozilla.org/en-US/security/faq/#ssl_config). Порівняння з тим, що реально віддає `ithillel.ua`:

Параметр	Mozilla Intermediate	ithillel.ua	Вердикт
Протоколи	TLS 1.2 + 1.3	TLS 1.2 + 1.3	збіг
Шифри TLS 1.3	AES-GCM x2 + ChaCha20	ті самі 3	збіг
Шифри TLS 1.2	лише ECDHE + AES-GCM/ChaCha20	+ 4 зайві CBC, без ChaCha20	розбіжність
HSTS	max-age ≥ 2 роки	1 рік, includeSubDomains, preload	коротший max-age; preload-статус окремо не перевірявся
Ключ	RSA 2048 достатньо	RSA 2048	збіг
OCSP stapling	у збереженому конфігу немає	вимкнено	не є розбіжністю ¹

¹ Let's Encrypt у 2025 припинив підтримку OCSP (лишилися тільки CRL), тож увімкнути stapling для цього сертифіката все одно не вийшло б.

Висновок: за параметрами, які можна перевірити зовні, сайт близький до Mozilla Intermediate, але має щонайменше дві розбіжності: список шифрів TLS 1.2 (зайві CBC і немає ChaCha20) та короткий HSTS `max-age`. Повну відповідність усьому nginx-конфігу зовнішнім скануванням довести не можна.

9. П'ять речень: що б я змінив у своєму сервісі

Мій «продакшн» — домашня мережа на Nest Wifi плюс пет-проекти за nginx, і ось що я б змінив після цього розбору.

1. У nginx вичистив би CBC-шифри з TLS 1.2, вставивши рядок `ssl_ciphers` прямо з генератора Mozilla, — рівно та розбіжність, яку знайшов у розділі 8.
2. Додав би HSTS `max-age=63072000; includeSubDomains` (а після перевірки субдоменів — і `preload`), бо без нього перший захід користувача по `http://` лишається вікном для downgrade-атаки.
3. Повісив би моніторинг терміну сертифіката (`openssl x509 -checkend` у cron з алертом за 14 днів): 72 дні з розділу 6 швидко стають нулем, якщо автопродовження мовчки зламалось.
4. На роутері вимкнув би відповіді на ICMP/закриті порти з WAN не чіпаючи ICMP типу 3/4 (fragmentation needed) — інакше зламається Path MTU Discovery, який розбирали в ДЗ 4.
5. Якщо порівняння WAN-адреси роутера з публічною адресою підтвердить CGNAT, замість відкривати порти назовні — тримати доступ до сервісів через WireGuard-тунель до VPS, а на VPS лишити відкритими лише 443 і порт WireGuard.

Звіт (підсумок)

Пункт	Результат
Сайт	ithillel.ua → 78.47.64.83 (Hetzner FSN1), AAAA є, DNS — Cloudflare
traceroute	12 хопів (ICMP), міжоператорський перехід на хопі 5 (~46 мс)
Handshake	кадри 1–3, RTT ≈ 39 мс; ClientHello з SNI=ithillel.ua — кадр 4
TLS	1.3, TLS_AES_256_GCM_SHA384, ServerHello у кадрі 8
curl -w	DNS 6 мс / TCP +38 / TLS +43 / TTFB +44; найдовше — тіло (~210 мс)
Сертифікат	Let's Encrypt YR2, лишилось 72 дні, ланцюжок повний (leaf + 2 проміжні)
SSL Labs	A+ (обидва ендпоінти); мінус — CBC-шифри в TLS 1.2
vs Mozilla	близько до Intermediate; відрізняються TLS 1.2 ciphers і HSTS max-age